Лабораторна робота №11
Тема: Робота з датою та часом.
Мета: Навчитися використовувати дату і час для виводу інформації

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Вбудований об'єкт `Date` містить дату і час, а також надає методи управління ними.

Наприклад, його можна використовувати для зберігання часу створення / зміни, для вимірювання часу або просто для виведення поточної дати.

Створення

Для створення нового об'єкта `Date` потрібно викликати конструктор `new Date()` з одним з таких аргументів:

- ***Без аргументів*** - створити об'єкт `Date` з поточними датою і часом:

```
let now = new Date();
alert( now ); // показує поточну дату і час
```

- Створити об'єкт Date з часом, що дорівнює ***кількості мілісекунд*** (тисячна частка секунди), що пройшли з 1 січня 1970 року UTC + 0.

```
// 0 відповідає  01.01.1970 UTC+0
let Jan01_1970 = new Date(0);
alert( Jan01_1970 );

// тепер добавимо 24 години і отримаємо 02.01.1970
UTC+0
let Jan02_1970 = new Date(24 * 3600 * 1000);
alert( Jan02_1970 );
```

- Створення об'єкта Date ***із заданими компонентами*** відбувається в місцевому часовому поясі.

Аргументи: Обов'язковими є лише перші два (year, month).

- Рік (`Year`): Повинен складатися з чотирьох цифр (наприклад, 2013). Використання двох цифр (наприклад, 98) призведе до інтерпретації року як 1998-го.
- Місяць (`Month`): Індексація починається з 0 (січень) до 11 (грудень).
- День (`date`): Параметр вказує на день місяця. Якщо його не задати, автоматично приймається значення 1.

Якщо параметри `hours/minutes/seconds/ms` відсутні, їх значенням стає 0. наприклад:

```
new Date(2011, 0, 1, 0, 0, 0, 0); // // 1 Jan 2011,
00:00:00
new Date(2011, 0, 1); // теж саме 0
```

Максимальна точність - 1 мс (до 1/1000 секунди):

```
let date = new Date(2011, 0, 1, 2, 3, 4, 567);
alert( date ); // 1.01.2011, 02:03:04.567
```

Методи отримання компонентів дати

Об'єкт Date у JavaScript надає ряд методів для отримання окремих частин дати (року, місяця тощо). Важливо пам'ятати, що всі ці методи повертають значення відповідно до **місцевого часового поясу**.

1. Основні компоненти (Рік, Місяць, Число)

- **`getFullYear()`** – повертає рік у форматі 4 цифр (наприклад, 2026).

Примітка: Ніколи не використовуйте застарілий метод `getYear()`. Він є нестандартним і часто повертає некоректні значення (наприклад, двозначний рік). Тільки getFullYear() гарантує точність.

- **`getMonth()`** – повертає порядковий номер місяця.

*Зверніть увагу***:** Відлік починається з **0** (січень) до **11** (грудень).

- **`getDate()`** – повертає день місяця (від 1 до 31).

Примітка: Попри назву `getDate`, метод повертає саме число (день), а не всю дату.

2. Компоненти часу

Ці методи повертають відповідні одиниці часу:

- **`getHours()`** – години (0–23).
- **`getMinutes()`** – хвилини (0–59).
- **`getSeconds()`** – секунди (0–59).
- **`getMilliseconds()`** – мілісекунди (0–999).

3. День тижня

`getDay()` –повертає день тижня у числовому форматі від **0 до 6**.

У JavaScript тиждень завжди починається з **неділі**, яка має індекс **0**. Субота, відповідно, має індекс **6**.

Всі перераховані вище методи повертають значення відповідно до місцевого часового поясу.

Однак існують і їх UTC-варіанти, які повертають день, місяць, рік для тимчасової зони UTC + 0: `getUTCFullYear()`, `getUTCMonth()`, `getUTCDay()`. Для їх використання потрібно після `get` додати `UTC`.

Якщо ваш місцевий часовий пояс зміщений відносно UTC, то наступний код покаже різні години:

```
// поточна дата
let date = new Date();
// час
alert( date.getHours() );
```

```
// час в часовому поясі UTC+0 alert( date.getUTCHours()
);
```

Крім вищенаведених методів, існують два особливих методу без UTC-варіанту:

`getTime()`

Для заданої дати повертає таймстамп - кількість мілісекунд, що пройшли з 1 січня 1970 року UTC + 0.

`getTimezoneOffset()`

Повертає різницю у хвилинах між місцевим часовим поясом і UTC:

Встановлення компонентів дати

Наступні методи дозволяють встановити компоненти дати і часу:

- `setFullYear(year, [month], [date])`
- `setMonth(month, [date])`
- `setDate(date)`
- `setHours(hour, [min], [sec], [ms])`
- `setMinutes(min, [sec], [ms])`
- `setSeconds(sec, [ms])`
- `setMilliseconds(ms)`
- `setTime(milliseconds)` (Встановлює дату у вигляді цілого кількості мілісекунд, що пройшли з 01.01.1970 UTC)

У всіх цих методів, крім `setTime()`, є UTC-варіант, наприклад: `setUTCHours()`.

Як ми бачимо, деякі методи можуть встановлювати відразу кілька компонентів дати, наприклад: setHours. Якщо якась компонента не вказана, вона не змінюється.

приклад:

```
let today = new Date();

today.setHours(0);
alert(today
today.setHours(0, 0, 0, 0);
alert(today);
```

Перетворення до числа, різниця дат

Якщо об'єкт Date перетворити в число, то отримаємо тайм стамп за аналогією з `date.getTime()`:

```
let date = new Date();
alert(+date); // кількість мілісекунд, те саме що
date.getTime()
```

Важливий побічний ефект: дати можна вичитати, в результаті отримуємо різницю в мілісекундах.

Цей прийом можна використовувати для вимірювання часу:

```
let start = new Date(); // починаємо відлік

// виконуємо певні дії
for (let i = 0; i < 100000; i++) {
  let doSomething = i * i * i;
}

let end = new Date(); // закінчуємо відлік дії

alert( `Цикл оброблено за ${end - start} мілісекунд` );
```

`Date.now ()`

Якщо потрібно просто виміряти час, об'єкт Date нам не потрібен.

Існує особливий метод Date.now(), який повертає поточну мітку часу.

Семантично він еквівалентний new Date().getTime(), однак метод не створює проміжний об'єкт Date. Так що цей спосіб працює швидше і не навантажує збирач сміття.

Даний метод використовується з міркувань зручності або коли важливо швидкодію, наприклад, при розробці ігор на JavaScript або інших спеціалізованих додатків.

Ймовірно, попередній приклад краще переписати так:

```
let start = Date.now(); // кількість мілісекунд з 1 січня
1970 року

// виконуємо певні дії
for (let i = 0; i < 100000; i++) {
  let doSomething = i * i * i;
}

let end = Date.now(); // закінчуємо відлік часу

alert( `Цикл оброблено за ${end - start} мілісекунд` );
// різниця числа, а не дати
```

Розбір рядка з датою: Метод `Date.parse(str)`

Метод `Date.parse(str)` призначений для зчитування дати з текстового рядка. Він повертає **таймстамп** – кількість мілісекунд, що минули з 1 січня 1970 року (UTC+0). Якщо рядок не відповідає формату, метод повертає `NaN`.

1. Стандартний формат (ISO 8601)

Для коректної роботи в усіх браузерах рекомендується використовувати формат:

`YYYY-MM-DDTHH:mm:ss.sssZ`

- **`YYYY-MM-DD`** – дата: рік-місяць-день.
- **`T`** — обов'язковий роздільник між датою та часом.
- **`HH:mm:ss.sss`** –час: години, хвилини, секунди та мілісекунди.
- **`Z`** – позначення часового поясу:
  - Символ `Z` означає нульовий пояс (UTC+0).
  - Формат `+hh:mm` або `-hh:mm` (наприклад, `+02:00` для України) вказує на зміщення відносно UTC.

2. Спрощені варіанти

Метод є гнучким і дозволяє опускати частини рядка:

- `YYYY-MM-DD` (Тільки дата)
- `YYYY-MM` (Рік та місяць)
- `YYYY` (Тільки рік)

3. Практичні приклади

наприклад:

```
let ms = Date.parse('2012-01-26T13:51:50.417-07:00');

alert(ms); // 1327611110417 (таймстамп)
```

Можна тут же створити об'єкт new Date з таймстамп:

```
let date = new Date( Date.parse('2012-01-26T13:51:50.417-07:00') );
alert(date);
```

КОРОТКО

Дата і час в JavaScript представлені об'єктом Date . Не можна створити «тільки дату» або «тільки час»: об'єкти Datезавжди містять і те, і інше.

Рахунок місяців починається з нуля (так, січень - це нульовий місяць).

Дні тижня в getDay()також відраховуються з нуля, що відповідає неділі.

Об'єкт Datесамостійно коригується при введенні значень, що виходять за рамки допустимих. Це корисно для додавання / віднімання днів / місяців / тижнів.

Дати можна вичитати, і різниця повертається в мілісекундах. Так відбувається, тому що при перетворенні в число об'єкт Date стає таймстамп.

Використовуйте Date.now() для швидкого отримання поточного часу в форматі таймстамп.

Врахуйте, що, на відміну від деяких інших систем, в JavaScript таймстамп в мілісекундах, а не в секундах.

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

***Завдання 1.* Створення дати**

1. Виведіть на екран поточні **день, місяць і рік** в форматі **'рік-місяць-день'**.

2. Виведіть на екран **поточний місяць словом**, українською мовою.

Завдання 2. **Робота з поточною датою**

3. Виведіть на екран **поточний день**.
4. Виведіть на екран **поточний місяць**.
5. Виведіть на екран **поточний рік**.
6. Виведіть на екран поточну дату-час в форматі **'12: 59: 59 31.12.2014 '**. Для вирішення цього завдання **напишіть функцію** , яка буде додавати **0** перед днями і місяцями, які складаються з однієї цифри (з **1.9.2014** зробить **01.09.2014**).

Завдання 3. **Робота з getDay**

7. Виведіть на екран **номер поточного дня тижня**.
8. Виведіть на екран **поточний день тижня** (словом, українською). Створіть для цього допоміжну функцію **showDay**, яка параметром приймає число, а повертає **день тижня українською** .
9. Дізнайтеся, який був **7-го січня 2025 року**.

Задання 4. **Робота з getTime**

10. Виведіть на екран кількість хвилин з **1-го січня 1970 року** до **теперішнього** моменту часу.

Завдання 5. **Робота з Date.parse**

11. Виведіть на екран **кількість годин** , що минув між **1 березня 1988 року** і поточним моментом за допомогою **Date.parse** .

Завдання 6. **Різниця між датами**

12. Виведіть на екран кількість секунд **з початку дня** до **теперішнього моменту** часу.
13. Виведіть на екран кількість секунд, яке залишилося **до кінця дня**.
14. Створіть **prompt**, в який користувач вводить дату свого народження в форматі **'2002-12-31'** (з конкретним роком). За втрати фокусу виведіть скільки днів залишилося до його дня народження. Скористайтеся методом Date.parse.
15. Визначити скільки років та місяців пройшло від ‘2024-10-10’ до сьогоднішньої дати.

Завдання 7. **Сучасна локалізація (Intl)**

16. Виведіть поточну дату у форматі: *"Сьогодні вівторок, 24 березня"*. Використовуйте метод `toLocaleString` або об'єкт `Intl.DateTimeFormat` з українською локалізацією (`'uk-UA'`), щоб отримати назву дня тижня та місяця без використання власних масивів.
17. Виведіть поточний час у форматі 24-годинного запису (наприклад, *"20:45"*), використовуючи налаштування об'єкта `options`.

Завдання 8. **Валідація та розумні дати**

18. Напишіть функцію `isValidDate(day, month, year)`, яка перевіряє, чи існує така дата насправді.

Підказка: Створіть об'єкт `Date`. Якщо введене число місяця або дня не збігається з тим, що зберіг об'єкт (наприклад, 31 лютого стало 2 березня), значить дата невалідна.

Перевірте роботу функції для значень: `(31, 1, 2026)` (лютий рахуємо від 0), `(29, 1, 2024)` та `(29, 1, 2025)`.

Завдання 9. **Маніпуляції та сетери (Setters)**

20. Створіть об'єкт `Date` з поточною датою. Використовуючи методи `set*`, змініть її наступним чином:

- Додайте до поточної дати **3 дні**.
- Від отриманого результату віднімiть **2 години**.
- Встановіть хвилини на **0**.